Keep local. Add cloud. Bring the tutor inside.

ChatGPT sign-in, embedded GPT/Gemini, and a low-cost hosting path for Hyperlearning / Kaizen.

Recommendation

Keep one shared application with separate **local** and **hosted** deployment modes. Start with a private hosted pilot, standard sign-in, and one AI provider. Treat ChatGPT plan access for the hosted product as an approval-dependent option, not a launch prerequisite.

1. ChatGPT sign-in: feasible, with an availability gate. The new feature exists. Website sign-in is a limited commercial-partner trial; open-source/local plan usage has a separate flow. Confirm which path Kaizen qualifies for before committing to it. [1][2][3]

2. In-app AI: very achievable. The app already builds lesson prompts, validates results, and schedules reviews. Add an in-app conversation and reuse those services rather than replacing the learning engine.

3. Hosting: not a rewrite, but not a deploy-button task. Next.js and Postgres are good foundations. The real work is account isolation, cloud file storage, safe imports, and keeping Anki connected to a desktop.

Scope	Planning estimate	What it means
Local AI proof of concept	4-7 engineer-days	One eligible ChatGPT-plan or API path; existing local app.
Private hosted app + tutor	2-4 working weeks	One allowed user; limited cloud scope; Anki writes stay local.
Multi-user hosted beta	8-13 working weeks	Tenant safety, embedded AI, Anki bridge, and launch hardening.

Estimates are engineering judgment, not quotes. One developer familiar with this repository; approval waiting time is excluded. Full scope and assumptions appear on page 8.

Reading guide: sign-in 2 | AI experience 3 | code audit 4 | architecture 5 | hosting 6 | AI costs 7 | effort 8 | AWS and launch gates 9 | sources 10-11

Sign-in is not the same as permission to use AI.

A verified account, an application session, and model usage are three separate concerns.

Option	What is available now	Implication for Kaizen
ChatGPT website SSO	OIDC + Authorization Code + PKCE. Selected commercial partners; a registered client is required. [2]	Moderate implementation effort once approved. Do not assume instant self-service access.
ChatGPT plan in a local / open-source app	Eligible Plus/Pro users can authorize plan usage. Dynamic registration is documented; paid/remotely hosted use has an interest process. [1][3][4]	A promising local pilot. Verify eligibility; private package metadata is not an open-source license.
Google sign-in	Standard Google identity integration is documented. [9]	Practical hosted fallback. Keep a provider-independent Kaizen user ID.
Gemini model access	API keys or project-backed OAuth; AI Studio subscription benefits are not a general third-party production API allowance. [10][11]	Use the Gemini API with explicit project billing or user-supplied API credentials.

The actual sign-in work

Use a maintained OIDC/session library. Validate signatures, issuer, audience, state and nonce; use PKCE and exact callbacks. Create a Kaizen session in secure cookies. Persist one-time transactions in shared storage, not instance memory. Link accounts explicitly, never just because two providers return the same email. [2]

Important boundaries

ChatGPT identity alone grants neither inference permission nor conversation access. Plan usage requires separate consent and consumes the user's existing allowance. Support disconnect, expiry and quota exhaustion without silently charging another billing source. [1][8]

A personal remote VM has its own documented credential-transfer path, but that is not evidence of permission to operate a general multi-user Vercel service. Hosted eligibility remains a decision gate. [7]

Replace copy-and-paste with a guided session.

Preserve the current learning rules and the manual workflow as a fallback.

Step	Proposed behavior
Prepare	Kaizen selects due/new grammar and daily vocabulary using the existing algorithm. Create an owned lesson with a stable ID.
Teach	Stream an in-app tutor conversation. Keep placement first, target grammar first, and secondary Japanese feedback afterward.
Rate	Offer the current Again / Good / Perfect controls. AI can propose ratings; the learner confirms before any learning-state change.
Save	Validate a typed result on the server, check ownership and scheduled item IDs, then call the existing transactional scheduling service.
Refresh	Update grammar, Daily, analytics and reading data only after a successful commit. Retries must not apply the lesson twice.

The visible grammar workflow already uses rating dropdowns; the older JSON importer still exists behind it. Preserve both. Do not reintroduce a requirement for the learner to paste model JSON. New-item Perfect and review-item Perfect must retain their different scheduling meanings. [R3]

Extend this later to tutor plans, speed-reading passages/questions and playlist image prompts; actual image generation stays separate.

Use a small provider adapter, not an autonomous agent

Separate lesson instructions from provider transport. Support OpenAI API first, eligible ChatGPT-plan access as its own adapter, and Gemini later. Both API families document structured outputs; validate semantics anyway, including refusals and incomplete responses. The model must never calculate authoritative SRS dates or execute arbitrary database commands. [19][20]

ChatGPT-plan preview needs special handling

Its HTTP flow requires streaming and store:false, with context supplied each turn; it is not identical to ordinary API-key requests. Discover account-supported models. Image generation and audio input are not supported on this route, so playlist artwork and future speaking practice need a separate supported API path. Several normal request fields, including max_output_tokens, are unavailable. [5][6]

Control cost with turn limits, bounded context, concurrency limits and an explicit provider/billing label. Keep tokens server-side or protected locally; never expose refresh tokens in browser storage. [24]

What actually needs to change for hosting.

Inspected the current working tree, including the recent YouTube work. This is a code review, not a deployment test.

Area	Observed in this codebase	Required change
Good foundation	Next.js 16.2.10, React 19, Prisma 7, Postgres and Zod; existing tests and reusable learning logic. [R1]	Retain these. No frontend or database-engine rewrite.
User isolation	49 models; most learning data has no user owner. Grammar, known words, books and playlists are global. Some services use user.findFirst(). [R2][R3]	Add ownership through direct fields or owned parents, tenant-scoped uniqueness, and authorization on every entry point.
Local Anki	Server calls 127.0.0.1:8765 for reviews, media and deck changes. [R4]	A cloud server cannot reach the learner's desktop this way. Use a local companion or disable live actions.
Files and imports	Playlist image upload writes into public/. Server Actions accept up to 25 MB. [R5][R6]	Private object storage and signed uploads. Vercel Functions have a 4.5 MB payload limit; increasing Next's limit does not override it. [14]
Assets and sync jobs	kuromoji loads a runtime dictionary. RTK JSON is now statically imported. External syncs/imports run in request-oriented code. [R6][R7]	Verify build packaging; review content distribution rights. Make large imports resumable, chunked jobs with progress.
Direct mutation route	YouTube progress writes by playlist ID without a session ownership check. [R8]	A login page alone is insufficient. Protect route handlers, Server Actions, media, and exports.

Reuse worth preserving: the grammar save path already validates data, applies scheduling transactionally, detects duplicate lesson results, and refreshes affected pages. Extend it with a learner identity rather than building a second scheduler. [R3]

Repository evidence locations are listed on page 11. No app files, schema, credentials, or hosted services were changed for this report.

Two deployment modes. One learning system.

A temporary development branch is useful; two permanent product forks are not.

Shared core	Local mode	Hosted mode
UI, curricula, SRS, analytics, prompt builders, validators	Current kaizen startup and local database remain usable.	Same screens and services, with authenticated user context.
Identity interface	Explicit local single-user identity; never enabled accidentally on the internet.	Provider accounts map to a stable Kaizen user and secure session.
File-storage interface	Local filesystem where appropriate.	Private object storage with tenant-owned keys and signed access.
Anki interface	Existing AnkiConnect adapter.	Owned snapshots plus a paired desktop companion.
AI interface	Copy prompt, API key, or eligible ChatGPT-plan connection.	App-paid API initially; approved user-plan access as an option.

Safest migration strategy

Back up the current database and media. Create a separate hosted database. Import an explicit snapshot into your owner account, backfill ownership, then validate totals. Test migrations on a copy first. Keep the local app and local database independent until you choose a cutover.

Once hosted, make the cloud the authority for cloud learning progress. The companion transfers Anki evidence; it should not attempt unrestricted bidirectional synchronization of two complete Kaizen databases. Full offline multi-device reconciliation is a separate project.

Anki stays on your computer

Start with manual snapshot import if speed matters. Later, a paired companion sends incremental reviews and selected media over outbound HTTPS. Use revocable device credentials, sequence checkpoints and per-user/card idempotency. Do not expose AnkiConnect publicly.

Initially make the companion read-only. Add deck injection and adaptive vocabulary-budget commands through a narrow allowlist with user consent, audit history, and safe retries. Show offline/last-sync status; the rest of the hosted app must work while Anki is closed.

Start managed.
Do not start with AWS complexity.

All figures are USD/month before tax. Published base prices are distinct from planning estimates.

Option	Published baseline	Fit for this application
Vercel + Supabase Recommended	Vercel Hobby $0 or Pro $20; Supabase Free $0 or Pro from $25. [12][15]	Best initial fit for Next.js plus managed Postgres and storage. Small non-commercial pilot can start free. Production baseline about $45, before overages.
Railway	Hobby $5 or Pro $20 minimum usage commitment; included usage is offset, not added twice. [16]	Good alternative for a conventional Node server and worker. Budget roughly $15-40 for a small personal app + DB as an unmeasured estimate, not a $5 guarantee.
AWS later	Fargate bills requested compute, memory and storage usage; other services add charges. [22]	Flexible control, but more operations work. Estimate only after choosing region, availability, DB size and networking. Not automatically cheaper.

What I would provision first

Vercel for the web app; Supabase for PostgreSQL and private file storage; a provider-neutral application session layer. Keep Prisma. Use the provider's supported pooled connection for app traffic and a separate migration connection. Do not assume database hosting automatically makes Prisma queries user-authorized. [23]

Free is appropriate for a pilot, not a promise

Vercel Hobby is for personal, non-commercial use. Supabase Free includes a 500 MB database and 1 GB file storage, may pause after a week of inactivity, and lacks the Pro automatic-backup offering. Large Anki review histories and media may exceed that quickly. [13][15]

Suggested budget: $0 infrastructure for an eligible tiny private trial; $25 with a paid database but personal Hobby frontend; about $45+ for Vercel Pro + Supabase Pro. Add AI, extra storage/egress, workers, domains and any extra environments. Recheck plan terms before public or paid use. [12][15]

AI cost depends on usage, not the sign-in button.

Illustrative text-only calculation; not a quality benchmark or a service quote.

Assumption: one active learner completes 30 sessions/month. Across all turns in each session, the app bills 20,000 input tokens and 5,000 output tokens: **0.60 million input + 0.15 million output per learner/month**. Include billed reasoning tokens in that output budget.

Illustrative model	Input / output per 1M tokens	AI / learner per month	AI / 100 learners
GPT-6 Luna [17]	$0.10 / $0.50	$0.135	$13.50
GPT-6.1 Sol [17]	$2.00 / $10.00	$2.70	$270.00
Gemini 3.1 Flash-Lite [18]	$0.25 / $1.50	$0.375	$37.50

Rates use standard paid text pricing; OpenAI short-context rates. Formula: (0.60 x input rate) + (0.15 x output rate). At 1,000 equally active learners, the AI-only examples become $135, $2,700 and $375 respectively. No caching or batch discount is assumed. [17][18]

Why the actual bill can be higher

Long conversations resend context. Retries, reasoning, larger lessons, more sessions and richer models change token totals. Images, audio, web search, storage and hosting are excluded. Add a 25% planning buffer for initial text usage, then replace estimates with measured cost per completed lesson.

Choose a funding model deliberately

App-paid API: simplest hosted pilot. You pay a project bill. Enforce a per-user daily/monthly budget and a global spend ceiling; do not rely solely on dashboard alerts.

User-supplied API key: optional later. The user funds model usage, but you still need encrypted credential storage, clear permissions and safe deletion. Local-only key storage is simpler.

ChatGPT plan: useful where eligible. It consumes the learner's existing usage, not a free new allowance. Preserve manual mode on quota exhaustion and ask before any paid API fallback. [8]

Evaluate inexpensive models against Japanese correction and placement examples before defaulting to them. For real learner data, review provider retention and data-use terms; Google's pricing distinguishes free-tier product-improvement use from paid-tier use. [18]

Most effort is product safety, not OAuth plumbing.

Engineer-days for one experienced developer. Estimates are not additive to the alternative pilot options below.

Full multi-user beta workstream	Days	Completion criterion
Eligibility / provider spike	1-2	One real sign-in and inference request on the intended deployment type.
Sessions + local/cloud boundaries	4-6	Safe login, logout, callbacks, identity mapping and local-mode safeguards.
Tenant data migration + authorization	7-12	Owned learning data, scoped uniqueness, secure actions/routes and isolation tests.
Hosting, storage, imports and jobs	4-7	Production build, backups, signed uploads, bounded/resumable import work.
One in-app AI tutor	5-8	Streaming, rating confirmation, validation, save/refresh and usage limits.
Anki companion	6-10	Pairing, incremental evidence, offline handling, safe command path.
Integration / security / release checks	4-7	Restore test, cross-user tests, failure handling and deployment runbook.

Total: 31-52 engineer-days, roughly 6-11 focused working weeks. Allow **8-13 weeks** for coordination, unexpected migration issues and stabilization. Provider-approval waiting time can be longer and is outside this estimate.

Smaller first steps

Local AI pilot: 4-7 days. One grammar experience and one eligible inference path, no multi-user conversion. **Private hosted pilot: 6-10 days without AI; 11-18 with a tutor.** One allowlisted account; retain only tested cloud-ready features; import snapshots manually and keep live Anki writes local.

Not included: paid subscriptions/payment processing, native mobile apps, unrestricted offline sync, voice tutoring, automatic artwork generation, legal review, or multiple AI providers at launch. A second provider is approximately 2-4 additional days after the common tutor interface works.

Move to AWS when there is a reason, not a user-count guess.

Vercel can grow with the app. AWS is an option, not an inevitable rewrite.

A portable path

Keep standard Postgres tables, provider-neutral storage and AI interfaces, and durable job records. Keep sessions out of process memory. Test a production Node/Docker build as a second deployment target; Next.js supports self-hosting, with additional cache and multi-instance coordination responsibilities. [21]

Today	Possible AWS destination	What changes
Vercel Next.js	ECS/Fargate containers	Deployment, load balancing, health checks and autoscaling.
Supabase Postgres	RDS PostgreSQL	Backup/restore migration, connection pooling and cutover.
Private object storage	S3 + signed access	Copy objects, switch adapter, validate media access.
Durable import/sync jobs	SQS + worker containers	Queue adapter and worker deployment; reuse job handlers.

Migrate for measured cost advantage, regional/compliance requirements, or worker/network needs. Start by moving workers if necessary, not the entire app. A managed AWS setup also pays for databases, load balancing, networking, logs and backups; container compute alone is not the total bill. [22]

Before inviting anyone else

Isolation: Two test users cannot read or mutate each other's lessons, Anki records, playlist progress, media, exports or analytics - including direct requests with another user's IDs.

Reliability: Duplicate saves apply once; interrupted AI sessions resume safely; oversized imports show progress; offline Anki does not block navigation. Dates use each learner's timezone.

Operations: Restore a backup, rotate/revoke credentials, verify usage caps, redact logs, and check production build assets. Review permissions to distribute imported RTK/course material; keep personal imports private.

The next decision: Choose a local AI pilot for subscription-powered tutoring, or a private cloud pilot for access from any device. Do not publish the current single-user build directly. No implementation has been started.

Official documentation and availability checks.

Sources checked 4 October 2026. Click a source number or URL. Availability, terms and prices can change.

[1] OpenAI: Sign in with ChatGPT quickstart
https://developers.openai.com/siwc/quickstart

[2] OpenAI: Website sign-in and availability
https://developers.openai.com/siwc/website

[3] OpenAI: ChatGPT plan usage for open-source apps
https://developers.openai.com/siwc/token-sharing-open-source

[4] OpenAI: Registration and sign-in
https://developers.openai.com/siwc/token-sharing-open-source/sign-in

[5] OpenAI: Models and inference
https://developers.openai.com/siwc/token-sharing-open-source/models-and-inference

[6] OpenAI: ChatGPT plan preview limitations
https://developers.openai.com/siwc/token-sharing-open-source/preview-limitations

[7] OpenAI: Self-hosted VMs
https://developers.openai.com/siwc/token-sharing-open-source/self-hosted-vms

[8] ChatGPT Learn: Plan usage and app limits
https://learn.chatgpt.com/docs/sign-in-with-chatgpt

[9] Google: OpenID Connect
https://developers.google.com/identity/openid-connect/openid-connect

[10] Google: AI plans versus Gemini API usage
https://ai.google.dev/gemini-api/docs/google-ai-plans

[11] Google: Gemini OAuth authentication
https://ai.google.dev/gemini-api/docs/oauth

[12] Vercel: Pricing
https://vercel.com/pricing

Interpretation: OpenAI documents both a general hosted-website trial and a separate open-source/local plan-usage flow. This report intentionally does not treat one as blanket authorization for the other. Google's published API/project path is the supported basis for the proposed Gemini integration.

The report's architecture choices, staffing ranges and workload projections are independent engineering estimates based on this repository, not commitments by these providers.

Pricing sources and repository evidence.

No credentials or personal learning records were needed for this assessment.

[13] Vercel: Hobby plan restrictions
https://vercel.com/docs/plans/hobby

[14] Vercel: Function limits
https://vercel.com/docs/functions/limitations

[15] Supabase: Pricing and included resources
https://supabase.com/pricing

[16] Railway: How subscription and usage billing work
https://docs.railway.com/pricing/understanding-your-bill

[17] OpenAI: API pricing
https://developers.openai.com/api/docs/pricing

[18] Google: Gemini Developer API pricing
https://ai.google.dev/gemini-api/docs/pricing

[19] OpenAI: Structured model outputs
https://developers.openai.com/api/docs/guides/structured-outputs

[20] Google: Structured outputs
https://ai.google.dev/gemini-api/docs/structured-output

[21] Next.js: Self-hosting
https://nextjs.org/docs/app/guides/self-hosting

[22] AWS: Fargate pricing
https://aws.amazon.com/fargate/pricing/

[23] Supabase: Prisma integration
https://supabase.com/docs/guides/database/prisma

[24] OpenAI: Accounts and sessions
https://developers.openai.com/siwc/token-sharing-open-source/profiles-and-sessions

Code inspected

[R1] /Users/milindc/gitrepos/Kaizen/package.json:1

[R2] /Users/milindc/gitrepos/Kaizen/prisma/schema.prisma:176

[R3] /Users/milindc/gitrepos/Kaizen/src/app/japanese/grammar/actions.ts:168

[R4] /Users/milindc/gitrepos/Kaizen/src/lib/anki-connect.ts:7

[R5] /Users/milindc/gitrepos/Kaizen/src/app/youtube/actions.ts:213

[R6] /Users/milindc/gitrepos/Kaizen/next.config.ts:3

[R7] /Users/milindc/gitrepos/Kaizen/src/lib/kanji-extract.ts:1

[R8] /Users/milindc/gitrepos/Kaizen/src/app/api/youtube/playlists/[id]/progress/route.ts:14